

SQL Murder Mystery [CORRECTION]

Can you find out whodunnit ?



Le projet **SQL Murder Mystery** a été créé par [Joon Park](https://twitter.com/joonparkmusic) (<https://twitter.com/joonparkmusic>) et [Cathy He](https://twitter.com/Cathy_MeiyingHe) (https://twitter.com/Cathy_MeiyingHe) lors de leurs travaux au Knight Lab de l'université américaine Northwestern University. Il a été adapté et développé pour le Web par [Joe Germuska](https://twitter.com/joegermuska) (<https://twitter.com/joegermuska>) pour en faire un exercice en ligne permettant de travailler la langage SQL.

Vous trouverez plus d'informations sur le [dépôt GitHub](https://github.com/NUKnightLab/sql-mysteries) (<https://github.com/NUKnightLab/sql-mysteries>) du projet.

Pour résoudre le meurtre, vous pouvez :

- le faire directement dans ce notebook SQL sur Capytale : la base de données `sql-murder-mystery.db` est attachée à ce notebook
- le faire en ligne à l'adresse : <https://mystery.knightlab.com/> (<https://mystery.knightlab.com/>)
- le faire en local (avec *DB Browser for SQLite* par exemple) en téléchargeant la base de données `sql-murder-mystery.db` attachée à ce notebook (ou en la téléchargeant sur le dépôt GitHub du projet en [cliquant ici](https://github.com/NUKnightLab/sql-mysteries/raw/master/sql-murder-mystery.db) (<https://github.com/NUKnightLab/sql-mysteries/raw/master/sql-murder-mystery.db>))

Énoncé original

Voici l'énoncé original :

*A crime has taken place and the detective needs your help. The detective gave you the crime scene report, but you somehow lost it. You vaguely remember that the crime was a **murder** that occurred sometime on **Jan. 15, 2018** and that it took place in **SQL City**. Start by retrieving the corresponding crime scene report from the police department's database.*

Le **schéma de la base de données** est le suivant :



Travail demandé

Vous rédigerez un rapport directement dans ce notebook dans lequel vous détaillerez les différentes étapes de votre investigation qui mènera je l'espère à l'identification du coupable :

- les **différentes idées** (dans des cellules de texte type Markdown ou Texte Brut)
- les **requêtes SQL effectuées** (dans des cellules de code, qui acceptent uniquement le langage SQL)
- les **déductions** que vous faites des résultats renvoyés par les requêtes (dans des cellules de texte type Markdown ou Texte Brut)

Par exemple, pour récupérer tous les enregistrements de la table `crime_scene_report`, on écrit la requête :

Entrée[]: `SELECT * FROM crime_scene_report;`

À vous de trouver le coupable !

Rédigez ci-dessous les différentes étapes de votre investigation. Ajoutez autant de cellule de texte et de code que nécessaire. Vous pourrez vérifier votre réponse un peu plus bas !

Étape 1 : On cherche les informations sur le crime en question :

Entrée[1]: `SELECT *
FROM crime_scene_report
WHERE date=20180115 AND type="murder" AND city = "SQL City";`

Sortie[1]:

date	type	description	city
20180115	murder	Security footage shows that there were 2 witnesses. The first witness lives at the last house on "Northwestern Dr". The second witness, named Annabel, lives somewhere on "Franklin Ave".	SQL City

On déduit de la description

Security footage shows that there were 2 witnesses. The first witness lives at the last house on "Northwestern Dr". The second witness, named Annabel, lives somewhere on "Franklin Ave".

des informations sur deux témoins.

Étape 2 : On cherche l'identité de deux témoins

Premier témoin :

```
Entrée[5]: SELECT *  
FROM person  
WHERE address_street_name = "Northwestern Dr"  
ORDER BY address_number DESC  
LIMIT 1;  -- optionnel
```

```
Sortie[5]:
```

id	name	license_id	address_number	address_street_name	ssn
14887	Morty Schapiro	118009	4919	Northwestern Dr	111564949

Deuxième témoin :

```
Entrée[6]: SELECT *  
FROM person  
WHERE name LIKE "Annabel%" AND address_street_name = "Franklin Ave";
```

```
Sortie[6]:
```

id	name	license_id	address_number	address_street_name	ssn
16371	Annabel Miller	490173	103	Franklin Ave	318771143

On vient donc de trouver nos deux témoins : **Morty Schapiro** (id = 14887) et **Annabel Miller** (id = 16371)

Étape 3 : On cherche leurs dépositions dans la table interview

Déposition de Morty Schapiro (id = 14887) :

Entrée[7]: **SELECT ***
FROM person
JOIN interview **ON** interview.person_id = person.id
WHERE person.id = 14887;

Sortie[7]:

id	name	license_id	address_number	address_street_name	ssn	person_id
----	------	------------	----------------	---------------------	-----	-----------

14887	Morty Schapiro	118009	4919	Northwestern Dr	111564949	14887
-------	----------------	--------	------	-----------------	-----------	-------

Déposition de Annabel Miller (id = 16371):

Entrée[8]: **SELECT ***
FROM person
JOIN interview **ON** interview.person_id = person.id
WHERE person.id = 16371;

Sortie[8]:

id	name	license_id	address_number	address_street_name	ssn	person_id
----	------	------------	----------------	---------------------	-----	-----------

16371	Annabel Miller	490173	103	Franklin Ave	318771143	16371
-------	-------------------	--------	-----	--------------	-----------	-------

Étape 4 : On utilise les dépositions pour trouver le suspect

Grâce aux informations fournies par Schapiro on cherche un membre "Gold" de la salle de sport *Get Fit Now Gym*. Voyons d'abord comment sont représentés les membres "Gold" :

Entrée[10]: **SELECT DISTINCT**(membership_status) **FROM** get_fit_now_member;

Sortie[10]:

membership_status
gold
regular
silver

On peut alors chercher la personne décrite par Schapiro :

Entrée[9]: **SELECT** p.id, p.name, p.license_id, p.ssn, gfnm.id **AS** gfnm_id
FROM get_fit_now_member **AS** gfnm
JOIN person **AS** p **ON** gfnm.person_id = p.id
JOIN drivers_license **AS** dl **ON** dl.id = p.license_id
WHERE gfnm.membership_status = "gold" **AND** gfnm.id **LIKE** "48Z%";

Sortie[9]:

id	name	license_id	ssn	gfnm_id
67318	Jeremy Bowers	423327	871539279	48Z55

On trouve qu'il s'agit de **Jeremy Bowers**.

Vérification des autres éléments

On peut vérifier sa plaque d'immatriculation :

```
Entrée[11]: SELECT dl.plate_number  
            FROM person AS p  
            JOIN drivers_license AS dl ON p.license_id = dl.id  
            WHERE p.id = 67318;
```

```
Sortie[11]:
```

<u>plate_number</u>
0H42W2

Il y a bien H42W dans le numéro de plaque !

On peut aussi vérifier que notre suspect Jeremy Bowers était bien à la salle de sport le 9 janvier 2018 comme l'indiquait Annabel Miller dans sa déposition :

```
Entrée[12]: SELECT *  
            FROM get_fit_now_check_in AS gfnci  
            WHERE gfnci.check_in_date = 20180109 AND gfnci.membership_id = "48Z55"
```

```
Sortie[12]:
```

<u>membership_id</u>	<u>check_in_date</u>	<u>check_in_time</u>	<u>check_out_time</u>
48Z55	20180109	1530	1700

Il était bien présent à la salle de sport le jour en question entre 15h30 et 17h00 !

Tout semble concorder ! Il reste à vérifier notre réponse !

Vérifiez votre réponse

Avez-vous trouvé le tueur ?

```
Entrée[13]: INSERT INTO solution VALUES (1, 'Jeremy Bowers');  
  
SELECT value FROM solution;
```

Sortie[13]:

value

Congrats, you found the murderer! But wait, there's more... If you think you're up for a challenge, try querying the interview transcript of the murderer to find the real villain behind this crime. If you feel especially confident in your SQL skills, try to complete this final step with no more than 2 queries. Use this same INSERT statement with your new suspect to check your answer.

Défi supplémentaire

Le défi supplémentaire peut se résoudre avec les deux requêtes suivantes.

On commence par chercher la transcription de la déposition de Jeremy Bowers comme indiqué :

```
Entrée[14]: SELECT i.person_id, i.transcript  
FROM interview AS i  
JOIN person AS p ON i.person_id = p.id  
WHERE p.id = 67318;
```

Sortie[14]:

person_id

transcript

67318	I was hired by a woman with a lot of money. I don't know her name but I know she's around 5'5" (65") or 5'7" (67"). She has red hair and she drives a Tesla Model S. I know that she attended the SQL Symphony Concert 3 times in December 2017.
-------	--

Il a été engagé par une femme riche. On cherche la femme en question avec les informations de la déposition de Jeremy Bowers :

```
Entrée[15]: SELECT p.id, p.name -- OU SELECT DISTINCT p.id, p.name
FROM drivers_license AS dl
JOIN person AS p ON dl.id = p.license_id
JOIN facebook_event_checkin AS fec ON fec.person_id = p.id
WHERE
    dl.gender = "female"
    AND 65 <= dl.height <= 67
    AND dl.hair_color = "red"
    AND dl.car_make = "Tesla" AND dl.car_model = "Model S"
    AND fec.event_name = "SQL Symphony Concert";
```

```
Sortie[15]:
```

	id	name
	99716	Miranda Priestly
	99716	Miranda Priestly
	99716	Miranda Priestly

On trouve qu'il s'agit de **Miranda Priestly**. Vérifions !

```
Entrée[16]: INSERT INTO solution VALUES (1, 'Miranda Priestly');

SELECT value FROM solution;
```

```
Sortie[16]:
```

	value
	Congrats, you found the brains behind the murder! Everyone in SQL City hails you as the greatest SQL detective of all time. Time to break out the champagne!

Références :

- Projet GitHub : <https://github.com/NUKnightLab/sql-mysteries> (<https://github.com/NUKnightLab/sql-mysteries>)
- Version Web : <http://mystery.knightlab.com/> (<http://mystery.knightlab.com/>)
- Crédits de l'image du détective de départ : [Vectors by Vecteezy](https://www.vecteezy.com/) (<https://www.vecteezy.com/>)

Germain Becker, Lycée Emmanuel Mounier, ANGERS. Licence [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/) (<https://creativecommons.org/licenses/by-sa/4.0/>)



